

SauronID — Technical Architecture

Verifiable agent binding. Cryptographic audit trails. Zero-trust runtime enforcement.

One-line summary

SauronID is a Rust HTTP service that issues, verifies, and audits cryptographically bound AI-agent action receipts, with TypeScript and Python SDKs for client integration, a Next.js dashboard for operators, Circom-based zero-knowledge circuits for compliance proofs, multi-vendor Trusted Execution Environment (TEE) attestation for tamper-proof runtime, and dual on-chain anchoring (Bitcoin via OpenTimestamps, Solana via the Memo Program) for tamper-evident audit trails.

System layers at a glance

Layer	Primary technology	Role
Backend service	Rust + axum + tokio	HTTP API, policy evaluation, cryptographic verification, audit anchoring
Persistent storage	SQLite + PostgreSQL	Agents, actions, policies, anchors, audit, multi-tenant state
Client SDKs	TypeScript (Node) and Python	In-process call signing, policy enforcement, stat submission
Frontend	Next.js (App Router) + React 19 + Tailwind	Operator dashboard, policy authoring, cohort views
Zero-knowledge proofs	Circom + circomlib + snarkjs (Groth16)	Compliance claims over committed action logs
On-chain anchoring	Bitcoin (OpenTimestamps) + Solana (Memo Program)	Public, tamper-evident timestamping
Privacy primitives	Custom Rust Differential Privacy + Paillier Homomorphic Encryption	Privacy-preserving benchmark aggregation
Trusted Execution Environments (TEE)	Multi-vendor attestation: AWS Nitro Enclaves, Intel SGX / TDX, AMD SEV-SNP, ARM CCA / TrustZone, TPM2, Apple Secure Enclave, Ed25519 software baseline	Tamper-proof, hardware-rooted runtime for the agent and the verifier
Cryptography	curve25519-dalek, ed25519-dalek, ring, sha2, hmac, subtle	Signatures, hashes, constant-time comparisons
Testing	Cargo test, Vitest, Pytest, custom redteam harness	Unit, integration, empirical adversarial suite
CI	GitHub Actions (test, audit, SBOM, security, release-gate, redteam-e2e)	Continuous verification

Backend service

Rust (2021 edition)

Rust is the language of the core service.

Why Rust:

- Memory safety without garbage collection: matters when handling cryptographic key material and request payloads from untrusted clients.
- Predictable performance: no GC pauses during signature verification under load.
- Strong type system catches a large class of authentication bugs at compile time.
- Excellent cryptography ecosystem (dalek family, ring, RustCrypto).
- Single statically linked binary simplifies deployment and supply-chain audit.

Alternatives considered: Go (easier hiring, weaker compile-time guarantees, weaker cryptography library story), C++ (too easy to make memory-safety mistakes in security-critical code), TypeScript/Node (unacceptable for a service that holds master signing material).

axum 0.8 (HTTP framework)

axum is the HTTP framework, built on top of tokio.

Why axum:

- First-class tower middleware ecosystem (CORS, tracing, rate limiting compose cleanly).
- Type-safe extractors that move authentication and tenancy checks into function signatures.
- Async-native, built by the tokio maintainers.
- Stable and widely adopted in the Rust web ecosystem.

tokio 1.49 (async runtime)

tokio is the async runtime — the de-facto standard in the Rust async ecosystem, required by axum, reqwest, and sqlx.

Persistent storage

SQLite (rusqlite 0.31, r2d2 connection pool)

SQLite is the default storage backend.

Why SQLite:

- Single-file deployment makes local development and small operators trivial to run.
- No network round-trip — every read is in-process.
- Bundled feature ships the SQLite engine inside the Rust binary, so the deployment has zero external dependencies.

PostgreSQL (sqlx 0.8)

PostgreSQL is the production storage backend.

Why PostgreSQL:

- Concurrent writes, replication, point-in-time recovery, and managed-service availability.
- Required for multi-tenant production deployments and for postgres-specific TOCTOU-safe operations.
- sqlx provides compile-time-checked queries against a live schema.

Backend selection is a single environment variable: `SAURON_DB_BACKEND=postgres`.

Client SDKs

TypeScript SDK (`agentic/`)

The TypeScript SDK is the primary client for Node.js agents.

Why TypeScript:

- The majority of LLM agent frameworks (LangChain JS, Vercel AI SDK) are in TypeScript.
- Browser-compatible builds possible from the same source.

Key dependencies:

- `@noble/ed25519` and `@noble/hashes` — audited, dependency-free pure-JS cryptography.
- `jose` — JWT signing and verification.
- `uuid` — nonce generation.

Python SDK (`clients/python/`)

The Python SDK targets data-science and LangChain Python users. Adapters for LangChain, OpenAI Assistants, and Anthropic Computer Use ship out of the box.

Why Python:

- The other half of the LLM agent ecosystem (LangChain Python, OpenAI Python, Anthropic Python).

- First-class integration with the dominant tool-calling frameworks.

Key dependencies:

- `requests` — HTTP client.
 - `cryptography` — audited cryptographic primitives.
-

Frontend

Next.js 16 (App Router) + React 19

Next.js is the frontend framework for the operator dashboard.

Why Next.js:

- Server components reduce the JavaScript bundle and let the dashboard render audit data without exposing internal API endpoints to the browser.
- Built-in API routes serve as a thin proxy to the Rust core, enabling auth and tenancy enforcement in one place.
- React 19's `use` and `cache` primitives fit the read-heavy dashboard workload.

Tailwind CSS 4

Tailwind is the styling system. Eliminates a CSS architecture problem for a small team and tree-shakes to a small CSS bundle.

Chart.js + react-chartjs-2

Chart.js renders the audit and cohort charts. Mature, batteries-included, sufficient for the dashboard's needs.

Other UI primitives

- `@radix-ui/react-*` for accessible primitives (dialog, dropdown, tabs, tooltip).
 - `@monaco-editor/react` for the policy YAML editor.
 - `pdf-lib` for generating audit reports as PDF.
 - `next-intl` for internationalization.
 - `next-themes` for dark mode.
-

Zero-knowledge proofs

Circom + circomlib + Groth16

Zero-knowledge circuits are written in Circom and compiled to Groth16 proving systems.

Why Circom:

- Most mature ZK DSL with the largest auditing and tooling ecosystem.
- circomlib provides battle-tested Poseidon, EdDSA, Merkle, and comparator components.
- Generated verifiers compile to Rust, Solidity, and JavaScript.

Why Groth16:

- Smallest proof size (constant ~200 bytes) and fastest verification.
- Production-grade prover and verifier tooling via snarkjs.

Circuits shipped: `MerkleInclusion`, `CredentialVerification`, `ActionRangeProof`, `ActionTimeWindow`, `ActionSetMembership`, `StatsHonestComputation`, and others. All circuits enable selective-disclosure compliance claims over committed action logs without revealing the underlying actions.

On-chain anchoring

Every action's Merkle root is anchored on two public chains for tamper-evident proof of existence.

Bitcoin via OpenTimestamps

OpenTimestamps is the Bitcoin anchoring backend.

Why Bitcoin + OpenTimestamps:

- Most-recognized public timestamping authority globally; legally accepted in many jurisdictions.
- OpenTimestamps is a free service that aggregates many submissions into a single Bitcoin transaction, removing per-anchor fees.
- The proof itself, once upgraded after Bitcoin confirmation, is verifiable forever without trusting OpenTimestamps.

Solana via Memo Program

Solana provides a second anchor with fast confirmation.

Why Solana:

- Sub-second finality for near-real-time anchoring.
- The Memo Program is a built-in, free system program that records arbitrary bytes on-chain — no deployed program required.
- Solana Explorer renders Memo transactions in a way that makes externally verifying an anchor trivial.

The repository also includes an optional custom Anchor program for operators who want richer on-chain semantics. Default deployment uses the Memo path.

Alternatives considered: Ethereum L1 (too expensive per transaction), Polygon / Arbitrum (added trust assumptions via sequencers), IPFS (not a timestamping service).

Privacy primitives

Differential Privacy

Differential Privacy noise is added to aggregate metrics before they are published in cohort views.

Implementation:

- Laplace and Gaussian mechanisms.
- Basic and Rényi composition for tight epsilon-budget accounting.
- k-anonymity suppression for small cohorts.
- Per-cohort epsilon-budget tracking with automatic rotation.

Why custom Rust: the mathematics is well-defined and small enough to implement in-house with high confidence, avoiding heavy Python-based DP frameworks and keeping the privacy boundary inside the Rust core.

Paillier Homomorphic Encryption

Additive homomorphic encryption supports privacy-preserving aggregation where customers submit encrypted contributions and the server computes the sum without ever decrypting individual values.

Why Paillier:

- Additive homomorphism is sufficient for sum-based aggregates.
 - Conceptually simple, well-studied since 1999.
 - Direct fit for cross-customer benchmark aggregation without third-party MPC nodes.
-

Trusted Execution Environments (TEE) and hardware attestation

A Trusted Execution Environment is a hardware-isolated region of a CPU that runs code sealed off from the host operating system, hypervisor, and even the machine owner. The chip cryptographically attests that a specific binary ran on genuine hardware with specific inputs and outputs. SauronID supports TEE-backed execution end-to-end: the customer's agent runs inside a TEE, the SauronID verifier runs inside a TEE, and every action receipt carries a hardware-rooted attestation that binds the agent binary, the policy, and the execution result to a specific CPU.

TEE backends supported:

Backend	Vendor	Typical deployment
AWS Nitro Enclaves	AWS	Confidential VMs on AWS EC2
Intel SGX (DCAP quotes)	Intel	On-premise or Azure Confidential Computing
Intel TDX	Intel	VM-level confidential compute (current generation)
AMD SEV-SNP	AMD	Azure, GCP, Oracle confidential VMs
ARM CCA / TrustZone	ARM	Edge, mobile, sovereign EU deployments
TPM2	TCG standard	Local hardware root for any platform
Apple Secure Enclave	Apple	iOS / macOS mobile clients
Ed25519 software baseline	—	Development and non-TEE deployments

What TEE attestation buys the customer:

- Cryptographic proof that the agent ran the exact audited binary, not a tampered version.
- Cryptographic proof that the policy enforcement code itself was not bypassed by a compromised host.
- A regulator-defensible "chain of custody" from CPU silicon → binary hash → action receipt → on-chain anchor.
- Independence from operator trust: even SauronID cannot forge an attestation; the hardware vendor's root certificate authority signs it.

Why multi-vendor:

- Different customers trust different hardware vendors (EU sovereignty, regulator preference, supply-chain risk modeling).
- Single-vendor lock-in is a deal blocker for compliance-sensitive customers in banking, healthcare, defense, and government.

- Customers elect the TEE backend that fits their risk model and existing cloud footprint at policy-attach time.

Attestation flow:

1. The agent or verifier binary is built with a published, reproducible hash.
2. At runtime, the TEE generates an attestation document signed by the vendor's hardware root key, embedding the binary hash, platform configuration, and a server-supplied nonce.
3. SauronID verifies the document against the vendor's root certificate authority, checks the binary hash matches the published value, and embeds the attestation handle in the action receipt.
4. Any third party — auditor, regulator, customer — can independently verify the attestation chain without trusting SauronID.

Dependencies: `ring`, `webpki`, `pem` (certificate-chain validation), CBOR parsing for vendor-specific attestation documents, vendor root CAs bundled at build time.

Cryptography primitives

The cryptography stack is built on audited, narrowly scoped Rust crates.

Primitive	Crate	Use
Ed25519 signatures	<code>ed25519-dalek</code> 2.1	Per-call signatures, A-JWT signing, attestation
Curve25519 scalar arithmetic	<code>curve25519-dalek</code> 4.1	Lower-level primitives, ring signatures
SHA-256 / SHA-512	<code>sha2</code> 0.10	Hashing, Merkle trees, agent checksums
HMAC	<code>hmac</code> 0.12	Symmetric authentication
Constant-time comparison	<code>subtle</code> 2.6	Timing-attack resistance
JWT	<code>jsonwebtoken</code> 9.3	A-JWT issuance and verification
Random	<code>rand</code> 0.8	Nonces, ephemeral keys
Argon2	<code>argon2</code> 0.5	Password hashing
X.509 / WebPKI	<code>webpki</code> 0.22, <code>ring</code> 0.17	Attestation cert-chain validation
Big integers	<code>num-bigint</code> 0.4	Paillier HE arithmetic
Base58	<code>bs58</code> 0.5	Solana address encoding

Why these specific crates:

- The `dalek` family is the most-audited Ed25519/Curve25519 implementation in Rust.
- `ring` is BoringSSL-derived and used in production by AWS, Cloudflare, and others.
- `subtle` enforces constant-time semantics at the type level.
- `jsonwebtoken` is the de-facto standard JWT crate.

Testing

Layer	Tooling
Rust core	<code>cargo test</code> — extensive integration coverage across policy, aggregation, multi-tenancy, DP, HE, and attestation
TypeScript SDK	Built-in Node test runner, custom end-to-end harness
Python SDK	Pytest
Frontend	Vitest + React Testing Library
End-to-end	Shell scripts covering payment, consent, KYA, delegation flows
Adversarial	Custom redteam harness with 40+ scenario files plus Tavily-driven autonomous attack fuzzer

Empirical suite: runs 16 concrete attack scenarios covering JWT replay, body mutation, nonce reuse, delegation bypass, revocation bypass, and config drift. All 16 attacks are blocked.

CI / Build

GitHub Actions workflows:

Workflow	Purpose
<code>test.yml</code>	Cargo + Node + Python test suites on every PR
<code>audit.yml</code>	<code>cargo audit</code> for known vulnerable dependencies
<code>sbom.yml</code>	Software Bill of Materials generation for each release
<code>security.yml</code>	Gitleaks, Trivy container scan, dependency review
<code>release-gate.yml</code>	Chains cargo test + quickstart + strict-mode smoke test
<code>redteam-e2e.yml</code>	Runs empirical attack suite

Build tooling: Cargo workspaces (Rust), tsc (TypeScript), setuptools (Python), Next.js build, Circom + snarkjs (ZK).

Top-level orchestration: `Makefile` provides `build`, `test`, `verify`, `demo`, `demo-strict`, `empirical`, `bench` targets.

What is deliberately not in the stack

A short note on technologies that are common in similar projects but were rejected by design:

Technology	Why not
Kubernetes	Service is a single Rust binary; ops complexity not warranted at current scale.
Kafka / queues	Synchronous request/response model is sufficient; anchoring batches use database polling, not a queue.
GraphQL	REST is enough; GraphQL would add a parsing surface.
Microservices	Splitting cryptographic state across services multiplies authentication surface; monolith intentionally preserved.
Solidity (Ethereum)	Considered for revocation; Solana anchoring is faster and cheaper for the use case.
LLM-generated ZK schemas	Considered and rejected — non-determinism and prompt-injection surface make this unsuitable for security-critical schemas.

How the layers connect

An agent — optionally running inside a customer-elected Trusted Execution Environment — uses the TypeScript or Python SDK to wrap its outbound tool calls. Before any tool runs, the SDK evaluates the bound policy locally and signs the call with the agent's Ed25519 key. The signed call hits the Rust core, which verifies the signature, evaluates the policy server-side, checks the TEE attestation, anchors the action into a Merkle tree, and periodically commits the root to Bitcoin (via OpenTimestamps) and Solana (via Memo). The Next.js dashboard reads from the same Rust core and displays the audit trail, policy state, and cohort benchmarks. Compliance disclosures are produced as Groth16 zero-knowledge proofs over committed action logs, verifiable by any third party with the proof and the on-chain root.

SauronID — Cryptographic agent binding for the AI-agent economy.